

Unstable Bluff Detection System

Dominic Ricci

Ryder Rocha

Dylan Hancock

Alex Hardy

Ryan Kotas

Project Description

Task:

The Unstable Bluff Detection System is a system to detect changes in the bluff holding up Del Mar, California's train tracks. If the bluff appears to be shifting or falling apart, the system will send an alarm to the proper authorities. The system serves as a warning system for potentially hazardous conditions, which could endanger passengers and cause damage to the railway.

Logistics:

The system works via 6 fixed cameras placed along the bluff. The cameras are given power and a Wi-Fi connection to a central server. The server contains imaging software and a database to store images up to a month old. As well as other images, once per day, up to a year old. According to the project description, the imaging software and database are externally handled.

Design:

For our part of the ecosystem, we designed two systems: The camera system and an interface system for the server. The camera system is present on each of the 6 cameras. It determines how the camera operates. It also sends information to the interface system. The interface system communicates with the already-handled imaging software and database.

Camera System

Externally, a time interval is set. Then, every time interval, the camera will take a photo. The camera will also collect a timestamp from the internet and its geo-locational coordinates from GPS. Then, it will send the image over the network to our server-side interface. As per the project description, the timestamp and locational data are stored in the image.

Server-Side Interface System

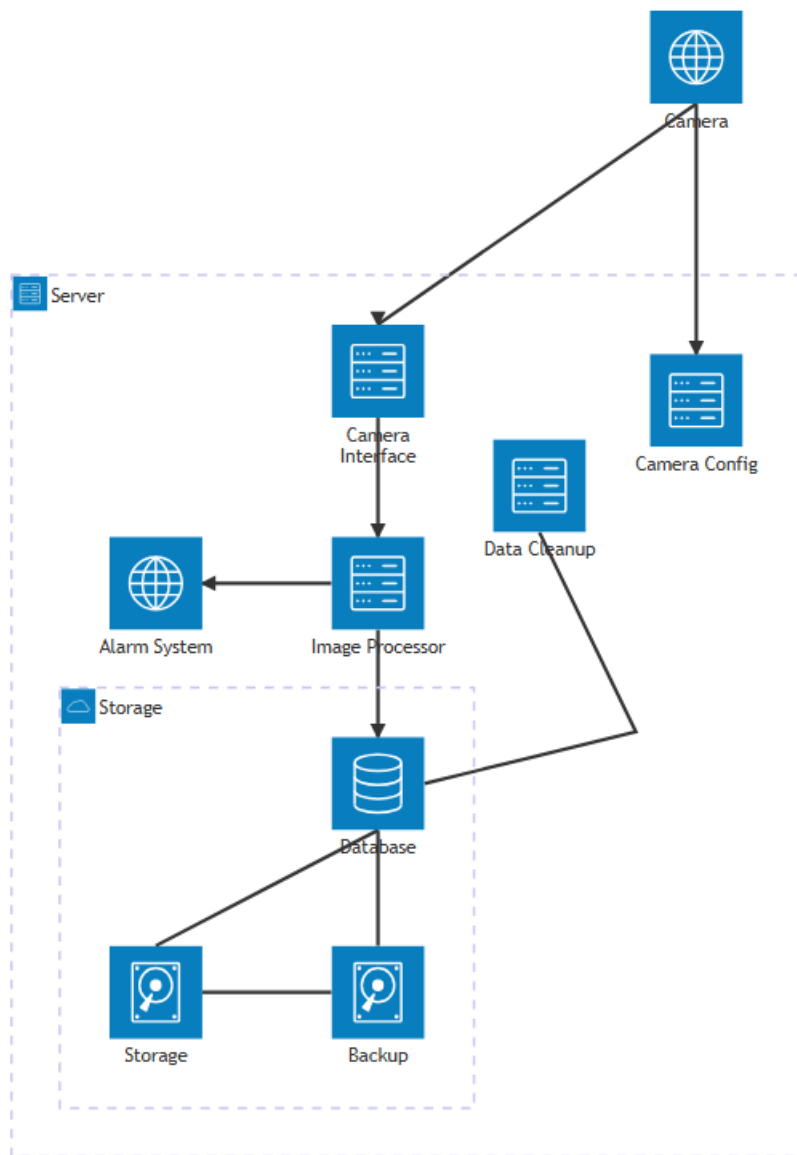
On the server side, we have three components: CameraListener, CameraConfig, and AlarmSystem. CameraListener waits for the camera to send image data. From there, the listener sends the data to the image software. The image software will then store the image in its database and determine whether it should send information about the image to the AlarmSystem. The AlarmSystem simply broadcasts information about the image to the authorities.

The CameraConfig determines the camera's resolution, time interval, and whether it takes pictures in color or greyscale. When a change is manually made to the config, it sends the updated information over the network to the camera. The camera then updates its local config based on this information.

Software Architecture Overview

For our architecture designs, we have a general software architecture design and a UML class design created in Visual Studio Code.

Software Architectural Diagram

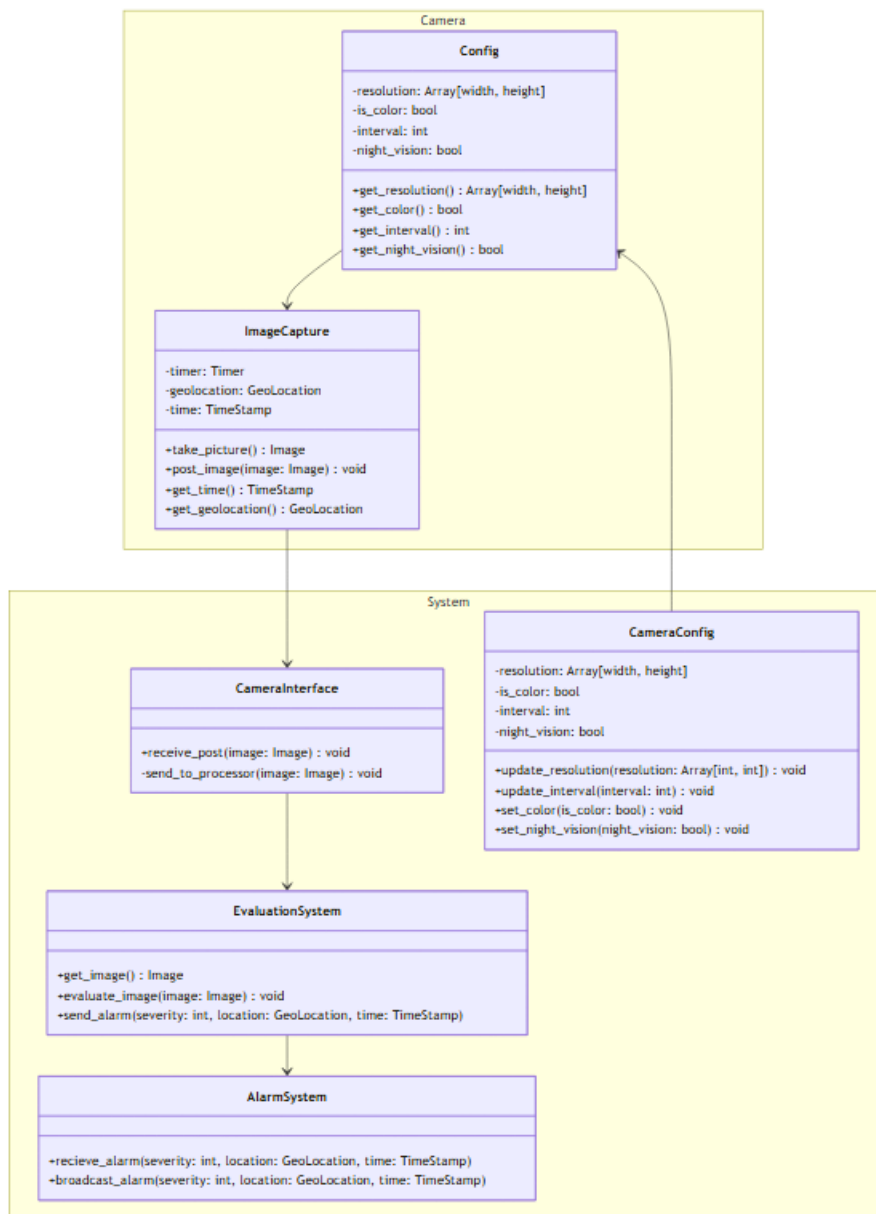


Software Architectural Description

This software architectural design has a central server that runs with a camera interface, camera configuration, the Image processor, and alarm system. The software system sends over server

post to the local database to store the images transmitted by the cameras. The image processor in the server will send an alert to the alarm system that notifies local authorities and Amtrak operators. The data cleanup all works locally to maintain essential images provided from the design specifications. The sections are with the cameras operating outside the server, and the storage of the database being located within the server at the central processing/testing center.

UML Class Diagram



Description of classes

- CameraInterface: A HTTP server that handles getting the image.
- CameraConfig: Class that handles the configuration of camera settings.
- Config: Class that receives that configurations of the camera settings
- EvaluationSystem: Class that evaluates the image on a scale of 0 to 5 with an image processing algorithm, and sends out an alarm if an image receives a level 5 rating.
- AlarmSystem: Class that signals the alarm in the event of a level 5 rating.
- ImageCapture: Class that captures the image and sends over the server to the CameraInterface.

Description of attributes

- resolution: a 2D integer Array configured by width by height (i.e. [width, height]) to store the value of resolution per image. Operator has choice to choose any desirable resolution to be set in array.
- is_color: a boolean value that checks if image is configured to be in-color or in black-and-white; if true, camera will take images in-color; if false, camera will capture images in black-and-white; value assumed to be default to true.
- night_vision: a boolean value that is set to true or false if the operator chooses to set the images to be captured with night vision; value assumed to be defaulted to false.
- interval: an integer value for the time interval between image capturing per camera; measurement assumption to be in seconds.
- timer: it will pull the updated interval to then update the time.
- geolocation: a GeoLocation provided from an image's 10-megapixels; in format of 32-bit geo location.
- time: a TimeStamp provided from an image's 10-megapixels; in format of 32-bit time stamp.

Description of operations

- get_resolution: a getter method that retrieves the resolution set to a camera and returns the array.
- get_color: a getter method that retrieves the boolean value of a camera regarding the color graphics that is either set to true or false and returns said value.
- get_interval: a getter method that returns the integer interval.
- get_night_vision: a getter method that returns the boolean value of a camera using night vision and returns said value.
- take_picture: the camera captures image when prompted after set interval is completed; returns image.

- `post_image`: the image is sent over through a server post to the `CameraInterface`; returns `void`.
- `get_time`: this getter method returns a `TimeStamp` from the image's bits.
- `get_geolocation`: this getter method returns a `GeoLocation` from the image's bits.
- `get_image`: the image is retrieved from the database and returned.
- `evaluate_image`: the image is retrieved to then be evaluated by the processing algorithm; returns `void`.
- `send_alarm`: in the case that an image receives a level of a four or five, the image's rating, geolocation, and timestamp is retrieved and sent to the `AlarmSystem`.
- `receive_alarm`: the alarm sent from the `EvaluationSystem` receives an image's severity rating, geolocation, and timestamp to be recorded in the system.
- `broadcast_alarm` operation: it receives the values sent by the evaluation system consisting of the level integer value indicated by the image processor algorithm, the 32-bit geo location, and the timestamp from the 10 Megapixels from an image. The image set off a rating of level four or five, indicating a major change or collapse has occurred. The alarm is then broadcasted out of the server towards the Amtrak railroad operators and local lifeguards to immediately stop train traffic and evacuate all human life from the location provided by the geolocation.
- `update_resolution`: it receives the requested resolution prompted by the operator and sets the given camera's resolution to new array value; returns `void`.
- `update_interval`: operator provides the new time interval for camera and the integer attribute interval is set to new value; returns `void`.
- `set_color`: uses boolean attribute `is_color` to alter a camera's image capturing mode by the operator's choice; returns `void`.
- `set_night_vision`: uses the boolean attribute `night_vision` to alter a camera's night vision mode by operator's choice; returns `void`.
- `receive_post`: image from `ImageCapture` is received by the `CameraInterface`; returns `void`.
- `send_to_processor`: image sent from `ImageCapture` is then sent to the processing algorithm; returns `void`.